B.TECH CCE REVISION · PART 7

Cloud, Distributed Systems and Reliability

Covers the marksheet subjects Cloud Computing and Software Reliability, plus the distributed-systems ideas behind both.

Isaqi · November 2026 · Written from general knowledge of the field as of late 2026; check tool versions, prices and vendor specifics against current documentation.

How to use this chapter

This is where scaling, automation and agentic systems actually run, and where things fail in ways that don't happen on a laptop: a call that times out but actually succeeded, a retry that charges a customer twice, a region that goes dark at midnight. The chapter builds the mental model in three steps: what the cloud is, why spreading work over many machines is hard, and how teams keep such systems reliable.

> **How to read it:** Sections 1–3 are the cloud, 4–6 are distributed systems, 7–10 are reliability and cost.
>
> - The labs use free tiers; set a billing alert before you deploy anything.

Contents

1. What "the cloud" is, and the service models
2. The building blocks
3. Containers, orchestration and CI/CD
4. Why distributed systems are hard
5. Consistency, CAP and its practical cousins
6. Idempotency, retries, timeouts and backoff
7. Reliability: availability, SLOs and error budgets
8. Observability: logs, metrics, traces
9. Fault tolerance and incident handling
10. Cost: what makes cloud bills blow up
11. Labs
12. Self-check questions and answers
13. Resources

1. What the cloud is

Renting someone else's computers by the second, with an API to create and destroy them, so you pay for use instead of owning hardware.

What changed with cloud computing wasn't the servers but the **API**: infrastructure became something you request in code and get in minutes, then release. That made it possible for a two-person startup to use the same infrastructure as a bank.

Model	You rent	You still manage	Examples
IaaS	Virtual machines, disks, networks	OS, patches, runtime, scaling, your app	AWS EC2, Google Compute Engine, Azure VMs
PaaS	A platform that runs your code	Your app and its config	Vercel, Render, Railway, Heroku, Google App Engine
Serverless / FaaS	Execution of individual functions per request	Just the function code	AWS Lambda, Cloudflare Workers, Vercel Functions
Managed services (BaaS)	A whole component, run for you	How you use it	Supabase, Firebase, managed Postgres, Auth0
SaaS	Finished software	Your data and settings	Gmail, Salesforce, Notion

The trade-off along the table: more control and lower unit cost at the top, less work and faster start at the bottom, at higher unit prices and more lock-in. **For an early product, buy as much as possible from the bottom of the table**; move down the stack only when cost or a specific limit forces it.

Shared responsibility: the provider secures the physical data centre, hardware and hypervisor; you secure what you put on it (configuration, access, data, code). Most cloud breaches are on the customer's side of that line: public buckets, leaked keys, over-broad permissions.

Regions and availability zones: a **region** (Mumbai `ap-south-1`, Hyderabad, Singapore) is a geographic area; each contains several **availability zones**, separate data centres with independent power and networking a few kilometres apart. Spreading across zones survives a data-centre failure; spreading across regions survives a regional disaster (at much higher cost and complexity). Data residency rules may require certain data to stay in India.

2. The building blocks

Every cloud offers the same dozen primitives under different names. Learn the primitives, not the product catalogue.

Block	What it is	AWS / GCP name	When you'd use it
Compute: VMs	A virtual server	EC2 / Compute Engine	Long-running services, full control
Compute: containers	Run container images, scaled for you	ECS, Fargate / Cloud Run	Most backends
Compute: functions	Code run per event, scales to zero	Lambda / Cloud Functions	Webhooks, light APIs, glue
Object storage	Unlimited files by key over HTTP	S3 / Cloud Storage	Audio, images, backups, exports
Block storage	A virtual disk attached to a VM	EBS / Persistent Disk	Databases on VMs
Managed database	Postgres/MySQL run for you, backups included	RDS, Aurora / Cloud SQL, AlloyDB	Almost every app
Queue	Hand work to be done later, reliably	SQS / Pub/Sub	Transcribe uploads, send emails, batch LLM jobs
Event bus / stream	Many consumers react to events	EventBridge, Kinesis / Pub/Sub; Kafka	Analytics pipelines, decoupling services
Cache	In-memory key-value	ElastiCache / Memorystore	Sessions, hot results, rate limits
Networking	Private networks, load balancers, DNS, CDN	VPC, ALB, Route 53, CloudFront	Everything
Identity (IAM)	Who and what can do which actions	IAM	Least privilege for people and services
Secrets and keys	Store and use secrets, encryption keys	Secrets Manager, KMS	Part 6
Scheduler	Run something on a timetable	EventBridge Scheduler / Cloud Scheduler	Nightly reports, cleanups

Queues deserve a closer look

A **queue** decouples "accept the work" from "do the work". When a student uploads an answer, the API stores the audio, puts a job on a queue, and returns immediately ("we're scoring your answer"). Workers pull jobs, transcribe, call the LLM, write scores. Benefits:

- The user isn't waiting on slow steps; the API stays fast.
- **Spikes are smoothed**: 5,000 uploads at 9 pm just make the queue longer; workers drain it.
- **Failures are retried** automatically; jobs that keep failing go to a **dead-letter queue** for inspection instead of being lost.
- Workers scale independently of the web tier.

Most queues deliver **at least once**, so a job may run twice. That's why workers must be idempotent (Section 6).

3. Containers, orchestration and CI/CD

Package the app once, run it anywhere, and let automation build, test and ship every change.

- **Containers** (Part 3) package the app and its dependencies into an image. The same image runs on a laptop, in CI and in production.
- **Orchestration**: something has to decide which machine runs which container, restart crashed ones, scale the count with load, roll out new versions gradually and route traffic. **Kubernetes** is the dominant orchestrator: powerful, and a lot of operational overhead. Managed container platforms (Cloud Run, Fargate, Render, Fly.io) give you most of the benefit without running it yourself. **A small team rarely needs Kubernetes.**
- **Infrastructure as code (IaC)**: describe infrastructure in files (Terraform, Pulumi, CloudFormation) instead of clicking in a console. Reviewable, repeatable, versioned, and the console-click mistakes go away.
- **CI/CD**:
- **Continuous integration**: every push builds the code and runs tests automatically (GitHub Actions, GitLab CI).
- **Continuous delivery/deployment**: passing builds are deployed automatically, often first to a **preview** or **staging** environment, then production.
- **Safe rollout patterns**:
- **Rolling**: replace instances a few at a time.
- **Blue-green**: run the new version alongside the old, switch traffic at once, switch back if it misbehaves.
- **Canary**: send 1–5% of traffic to the new version, watch metrics, then expand.
- **Feature flags**: ship code turned off; turn it on for some users without a deploy; turn it off instantly if it breaks.
- **Twelve-factor ideas** worth keeping: config in environment variables, stateless processes (state lives in databases and object storage, so any instance can serve any request), logs as streams, the same build promoted through environments.

4. Why distributed systems are hard

As soon as work spans more than one machine, you lose three things you took for granted: that things either work or don't, that messages arrive promptly, and that everyone agrees what time it is.

- **Partial failure**: one machine, disk, network link or service fails while others work. On a single computer, a crash takes everything down together, which is simple. In a distributed system, something is always broken somewhere.
- **Unreliable networks**: a message can be lost, delayed, duplicated or reordered. Crucially, **when a request times out, you don't know whether it was processed.** The server may have charged the card and the reply was lost.
- **No shared clock**: each machine's clock drifts. You can't reliably order events across machines by timestamp alone; "last write wins" by timestamp can silently drop data.
- **Fallacies of distributed computing** (Peter Deutsch and colleagues): the network is reliable; latency is zero; bandwidth is infinite; the network is secure; topology doesn't change; there is one administrator; transport cost is zero; the network is homogeneous. Each false assumption is a source of bugs.
- **Consensus**: getting several machines to agree on one value (who is the primary database? has this transaction committed?) despite failures is a hard problem solved by algorithms like **Raft** and **Paxos**. You'll rarely implement them, but they're inside etcd (Kubernetes' brain), CockroachDB, Spanner and many managed services.

Worked example: the double charge

A checkout service calls the payment provider; the call times out after 10 s. If it doesn't retry, the payment may have failed and the order is lost. If it retries blindly, the payment may have succeeded and the customer is charged twice. The correct answer is neither: send an **idempotency key** (Section 6) so the provider recognises the retry and returns the original result.

5. Consistency, CAP and its practical cousins

When data is copied across machines, you choose how stale a read may be, and what happens during a network split.

- **Strong consistency (linearisability)**: every read sees the latest write, as if there were one copy. Simple to reason about, costs latency and availability.
- **Eventual consistency**: replicas converge if writes stop; meanwhile reads may be stale. Fast and available. DNS, CDNs, many NoSQL stores and social media like-counts work this way.
- Useful middle grounds: **read-your-writes** (you always see your own updates), **monotonic reads** (you never see time go backwards), **causal consistency** (replies never appear before the messages they reply to).

CAP, in plain terms: during a **network partition** (some machines can't talk to others), a system must choose between **consistency** (refuse some requests rather than risk stale or conflicting data) and **availability** (answer everything, possibly with stale data, reconcile later). Partitions are a fact of life, so the real choice is C vs A *when they happen*.

PACELC adds the everyday case: **E**lse (no partition), you trade **L**atency against **C**onsistency, because keeping replicas in sync means waiting for them.

Data	Sensible choice
Wallet balance, bookings, inventory	Strong consistency; reject or retry rather than oversell
Leaderboards, view counts, feeds	Eventual is fine
A user's own profile edits	Read-your-writes
Analytics dashboards	Minutes stale is fine

6. Idempotency, retries, timeouts and backoff

In a world of timeouts and duplicate messages, make operations safe to repeat, and retry politely.

- **Idempotent** means doing it twice has the same effect as doing it once. `SET status = 'paid'` is idempotent; `balance = balance - 100` isn't.
- **Idempotency keys**: the client generates a unique key per logical operation (`order-8812-charge`) and sends it with each attempt; the server stores the key with the result, and on a repeat returns the stored result instead of acting again. Payment APIs (Stripe, Razorpay) support this; your own APIs and queue workers should too.
- **Timeouts on every network call**, shorter than the user's patience and the caller's own timeout. No timeout means a hung dependency hangs you.
- **Retry only what's safe**: idempotent operations, or those with keys. Retry transient failures (timeouts, 429, 502, 503, 504), not 400s.
- **Exponential backoff with jitter**: wait 1, 2, 4, 8 s... plus a random amount, up to a cap and a maximum number of attempts. Without jitter, thousands of clients retry in lockstep and knock the recovering service over again (a **retry storm**).
- **Retry budgets**: if every layer retries 3 times, a 4-layer system turns one failure into 81 attempts. Retry at one layer, ideally the one closest to the user.
- **Circuit breakers**: after repeated failures to a dependency, stop calling it for a while and fail fast (or serve a fallback), then probe to see if it's back. Protects both sides.
- **Bulkheads**: separate pools (connections, threads, rate limits) for different dependencies, so one slow dependency can't use up everything.

```
import random, time

class Transient(Exception): pass

def call_with_retries(fn, attempts=5, base=0.5, cap=8.0):
    for i in range(attempts):
        try:
            return fn()
        except Transient:
            if i == attempts - 1:
                raise
            delay = min(cap, base * 2 ** i) * random.uniform(0.5, 1.5)  # backoff + jitter
            time.sleep(delay)
```

7. Reliability: availability, SLOs and error budgets

Reliability is a feature with a cost. Decide how much you need, measure it from the user's point of view, and spend the rest on shipping.

The nines

Availability	Downtime per year	Per 30 days
99% ("two nines")	~3.65 days	~7.2 hours
99.9%	~8.8 hours	~43 minutes
99.95%	~4.4 hours	~22 minutes
99.99%	~53 minutes	~4.3 minutes
99.999%	~5 minutes	~26 seconds

Each extra nine costs roughly ten times more effort. **Dependencies multiply**: if your request path needs five services each at 99.9%, the whole is at most about 99.5%. You can't be more reliable than the things you depend on without redundancy.

SLIs, SLOs, SLAs

- **SLI** (indicator): a measurement of user experience, e.g. "share of score requests answered successfully within 3 s".
- **SLO** (objective): the target for it, e.g. "99.5% over 30 days". Internal.
- **SLA** (agreement): a contractual promise to customers with penalties; set looser than the SLO.
- **Error budget**: 100% minus the SLO. At 99.5%, about 3.6 hours of "bad" per month is allowed. If you're well within budget, ship faster and take risks; if the budget is spent, slow down and fix reliability. It turns "reliability vs features" from an argument into a number.

Good SLIs measure what users feel (successful requests, latency at the **p95/p99**, not the average), at the edge of the system, not CPU usage on one server. Averages hide pain: a 400 ms average can coexist with 5% of users waiting 8 seconds.

Software reliability, the classical view

The marksheet's Software Reliability subject treats reliability statistically: **MTBF** (mean time between failures), **MTTR** (mean time to repair), availability = MTBF / (MTBF + MTTR), failure-rate curves (the **bathtub curve**: early failures, a stable period, wear-out), and reliability growth models that predict defects remaining from the rate they're being found. The modern lesson from those formulas: **reducing MTTR (detect and recover fast) is usually cheaper than increasing MTBF (never fail)**, which is why observability, rollbacks and feature flags matter so much.

8. Observability: logs, metrics, traces

You can only fix what you can see. Three complementary signals, each answering different questions.

Signal	What it is	Answers	Tools
Logs	Timestamped records of events, ideally structured (JSON)	What exactly happened to this request?	Logtail/Better Stack, Datadog, CloudWatch, Loki
Metrics	Numeric time series, aggregated	How many, how fast, how often, trending how?	Prometheus + Grafana, Datadog, CloudWatch
Traces	The path of one request through every service, with timings	Where did the 4 seconds go?	OpenTelemetry, Jaeger, Honeycomb, Sentry

Practical rules:

- Give every request a **request ID** and include it in every log line and downstream call, so you can follow one user's failure across services.
- Log **structured fields** (`user_id`, `route`, `status`, `duration_ms`, `model`, `tokens`), not prose sentences.
- **Never log secrets or sensitive personal data** (Part 6).
- Watch the **four golden signals** (Google SRE): **latency, traffic, errors, saturation**. For each service, a dashboard with those four answers most "is it broken?" questions.
- **Alert on symptoms users feel** (error rate, latency SLO burn), not on every cause (one CPU at 90%). Every alert should be actionable; noisy alerts get ignored.
- **Error tracking** (Sentry) groups exceptions with stack traces and counts, which catches bugs before users report them.
- For LLM features, log per call: model, prompt version, input/output tokens, latency, cost, and outcome. LLM-specific observability tools (Langfuse, LangSmith, Helicone) make this easy.

9. Fault tolerance and incident handling

Design so that failures are contained, degrade gracefully, and are recovered from quickly.

- **Redundancy**: no single point of failure for anything critical: multiple instances behind a load balancer, database with a standby replica in another zone, backups in another region.
- **Health checks**: load balancers and orchestrators send traffic only to instances that pass; failed ones are replaced.
- **Graceful degradation**: when a dependency fails, do something useful instead of failing completely. If the LLM provider is down, accept the recording and score it later ("results in 10 minutes") rather than erroring; fall back to a second model provider; show cached results.
- **Load shedding**: under overload, reject some requests quickly (429/503) so the rest succeed, rather than everyone timing out.
- **Backups and restore drills**: a backup you've never restored is a hope, not a backup. Know your **RPO** (how much data you can lose: last hour? last minute?) and **RTO** (how long to recover).
- **Chaos engineering**: deliberately break things in controlled ways (kill an instance, add latency) to find weaknesses before real failures do.

Incidents

1. **Detect** (alerts, users), **declare** an incident, and assign one **incident commander** who coordinates.
2. **Mitigate first, diagnose later**: roll back the last deploy, turn off the feature flag, fail over, scale up. Restoring service beats understanding why.
3. **Communicate**: status page and updates on a regular schedule.
4. **Blameless post-mortem**: timeline, impact, root causes and contributing factors, what went well, action items with owners. Focus on how the system allowed the mistake, not who made it; otherwise people hide mistakes.

10. Cost: what makes cloud bills blow up

Cloud is cheap to start and easy to overspend. Most surprise bills come from a short list.

Culprit	How it happens	Guard
Forgotten resources	Test VMs, databases, GPU instances left running	Tags, budgets, regular cleanup, scale-to-zero platforms
Data transfer (egress)	Serving large files directly from storage or across regions; cross-zone chatter	CDN for media, keep services in one region, watch egress line items
Over-provisioning	Big instances "to be safe"	Right-size from real metrics; autoscale
Managed service markups	NAT gateways, managed load balancers, log ingestion priced per GB	Know the unit prices of what you enable
Runaway serverless or loops	A bug triggers functions recursively, or a retry storm	Concurrency limits, budget alerts
LLM API usage	Long prompts, large contexts on every call, no caching, abuse	Per-user quotas, prompt caching, smaller models for simple steps, batch APIs for non-urgent work
Logs and metrics	Logging everything at debug level	Sample, set retention, drop noisy logs

Two habits catch most of it: a **budget alert** on every account from day one, and computing **unit economics** (cost per active user, cost per scored session) so you notice when they drift. For LLM features, cost per session is often the biggest number: `(input tokens × input price + output tokens × output price) × calls per session`. Check current per-token prices from the provider; they change often.

11. Labs

Lab 1. Deploy something small, observably. Deploy a tiny API (FastAPI or Express) to a PaaS with a free tier (Render, Railway, Fly.io, Vercel Functions, or Cloud Run). Add one structured log line per request with a request ID and duration. Set up an uptime check (Better Stack, UptimeRobot) and a billing alert. Then break it (deploy a version that crashes on one route) and use only the logs to find out what's wrong.

Lab 2. Retries done right. Run the backoff function from Section 6 against a fake service that fails 70% of the time:

```
import random
calls = 0
def flaky():
    global calls; calls += 1
    if random.random() < 0.7:
        raise Transient("503")
    return "ok"
print(call_with_retries(flaky, attempts=8, base=0.05), "after", calls, "calls")
```

Then: add an idempotency key and a dictionary acting as the server's "already processed" store, and make a "charge" operation that runs at most once however many times it's retried.

Lab 3. A queue in miniature. Using Python's `queue.Queue` and two worker threads, process 20 fake "transcription" jobs where some fail. Retry failed jobs up to 3 times, then move them to a dead-letter list. Print the final state.

Lab 4. Write an SLO. For a product you're building, define 2 SLIs, SLO targets, the resulting monthly error budget in minutes, and what you'd alert on. One page.

Lab 5. Cost model. In a spreadsheet, estimate monthly cost for 1,000 and 10,000 active users: hosting, database, storage for audio (MB per session × sessions), transcription, LLM tokens per session, egress. Find the line that dominates.

12. Self-check questions

1. **IaaS vs PaaS vs serverless, one line each?** Rent VMs and run everything yourself; push code to a platform that runs it; supply functions that run per event and scale to zero.
2. **What's the shared responsibility model?** The provider secures the infrastructure; you secure your configuration, access, code and data.
3. **Why put a queue between upload and scoring?** Fast responses, smoothed spikes, automatic retries, independent scaling, and nothing lost when a step fails.
4. **Why does a timeout not tell you whether the request succeeded?** The request may have been processed and only the reply lost.
5. **What is idempotency and how do you get it for a payment?** Repeating has the same effect as doing once; send a unique idempotency key so retries return the original result.
6. **Why add jitter to backoff?** So many clients don't retry at the same moments and overwhelm a recovering service.
7. **CAP in one sentence?** During a network partition, a replicated system must choose between staying consistent (refusing some requests) and staying available (possibly serving stale data).
8. **Where would you accept eventual consistency?** Counts, feeds, leaderboards, analytics; not balances or bookings.
9. **What does 99.9% availability allow per month?** About 43 minutes of downtime.
10. **What is an error budget for?** It quantifies allowed unreliability, so the team knows when to ship faster and when to stop and fix reliability.
11. **Why alert on p99 latency rather than average?** Averages hide the slow tail that a meaningful share of users actually experience.
12. **Logs vs metrics vs traces?** What happened to a specific request; aggregated numbers over time; the timed path of one request across services.
13. **The LLM provider is down. What should the product do?** Degrade gracefully: queue the work and score later, fail over to another model, or show cached results with a clear message.
14. **Why are post-mortems blameless?** Blame makes people hide mistakes; the goal is to fix the system that allowed them.
15. **Name three common causes of surprise cloud bills.** Forgotten resources, data egress, and unbounded usage (serverless loops, LLM tokens, log volume).

13. Resources

Check these are still available before committing time.

- **Google, *Site Reliability Engineering*** and ***The Site Reliability Workbook*** (free at sre.google). Read the chapters on SLOs, monitoring distributed systems, handling overload and post-mortems.
- **Martin Kleppmann, *Designing Data-Intensive Applications***, chapters 8 and 9 (the trouble with distributed systems; consistency and consensus).
- **MIT 6.5840 Distributed Systems** (lectures and papers online) if you want the deep version.
- **AWS Well-Architected Framework** (free). Six pillars, a good checklist for any cloud design.
- **A cloud provider's foundational certification syllabus** (AWS Cloud Practitioner, Google Cloud Digital Leader) as a structured tour of services; the exam itself is optional.
- **Marc Brooker's blog** (brooker.co.za) on retries, backoff and jitter, and the **Amazon Builders' Library** articles on timeouts, retries and idempotency.